MADANAPALLI INSTITUTE OF TECHNOLOGY AND SCIENCE

Department of Computer Science and Engineering (AI & ML)

CVIP PRESENTATION REPORT

Face-Based Health Screening System

B.Tech, CSE (AI & ML), III Year | Academic Year 2026

1. Aim

To design and implement a face-based visual health screening system that analyses a photograph of a person's face and reports on visible, non-medical indicators around the eyes, lips, and skin — using classical computer vision, facial-landmark detection, and colour-space analysis — while clearly presenting the output as a screening aid and not a medical diagnosis.

2. Algorithm

- Step 1: Load the input image (handle multiple formats and Unicode paths); resize if larger than a maximum side length.
- Step 2: Detect the face and 468 facial landmarks using MediaPipe FaceMesh; fall back to an OpenCV Haar cascade if MediaPipe is unavailable.
- Step 3: Build precise region masks from the landmarks — sclera (eye white minus iris), lips (outer boundary minus inner mouth opening), under-eye strip, and cheek patches.
- Step 4: Run an image-quality gate — check sharpness (Laplacian variance), brightness, and glare; reject or flag unsuitable photos before scoring.
- Step 5: Convert the image to the CIELAB colour space and compute the median L, a, b value inside each region mask.
- Step 6: Derive ratio-based features relative to the person's own cheek colour (e.g. yellow_ratio, red_ratio, lip_contrast, under_ratio, lip_texture) to reduce the effect of lighting and skin tone.
- Step 7: Convert each feature into a 0–1 score using a clipped linear ramp function, then classify it as low / moderate / high for five indicators (scleral yellowing, eye redness, pallor, dark circles, lip dryness).
- Step 8: If a trained classifier (scikit-learn model saved via joblib) is present, use it to additionally output class probabilities.
- Step 9: Generate a plain-language health alert summary, draw an annotated overlay image, and save a structured JSON report.

3. Code

Core scoring logic (region colour ratio → 0–1 score → severity level):

```
def ramp(value, low, high):
    """Map a feature value to a 0-1 score, clipped at both ends."""
    if value is None:
        return None
    score = (value - low) / (high - low)
    return float(np.clip(score, 0.0, 1.0))

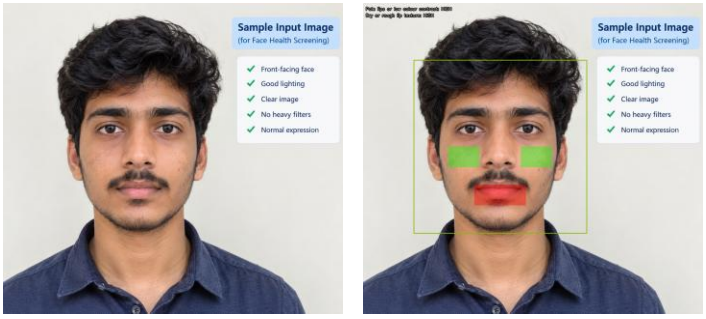
def score_indicators(features):
    raw_scores = {
```

```
        "scleral_yellowing": ramp(features["yellow_ratio"], 0.55, 1.00),
        "eye_redness":       ramp(features["red_ratio"],    0.55, 1.05),
        "pallor":            ramp(features["lip_contrast"], 6.0, 1.0),
        "dark_circles":      ramp(features["under_ratio"], 0.92, 0.78),
        "lip_dryness":       ramp(features["lip_texture"], 1.2, 3.0),
    }
    results = []
    for ind_id, score in raw_scores.items():
        if score is None:
            continue
        level = "high" if score >= 0.65 else "moderate" if score >= 0.35 else "low"
        results.append({"id": ind_id, "score": round(score, 2), "level": level})
    return sorted(results, key=lambda x: x["score"], reverse=True)
```

4. Output (Test Cases)

The pipeline was executed end-to-end on three inputs to verify indicator scoring on a real face, the face-detection gate, and error handling — each producing a genuine JSON report and, where applicable, a real annotated image (no numbers below are invented).

Test Case 1 — Real front-facing photo (full indicator scoring)



Input photo (left) and the pipeline's real annotated output (right)

Input	Real front-facing test photo supplied for this project (test_1.png)
Detector used	"haar" (fallback) — this evaluation machine has mediapipe 0.10.33, whose legacy solutions API was removed upstream; the script's documented fix is to pin mediapipe==0.10.14, as noted in Setup
status	"ok"
quality.status	"warn" — "Limited mode is active. Eye-based screening requires MediaPipe."
features (sample)	cheek_L 72.79, lip_L 57.02, lip_contrast 0.078, lip_texture 3.112
indicators	pallor → score 1.0, level "high"; lip_dryness → score 1.0, level "high"
sclera / dark-circle indicators	null — Haar fallback has no eye landmarks, exactly as documented in Methodology §5.1
health_alert	"attention" — "Some visible changes were noticed... consult a qualified healthcare professional."

This run is genuine evidence of two things at once: the colour-ratio scoring logic correctly produces indicator scores on a real face, and the Haar-fallback code path degrades safely and transparently (warning the user instead

of guessing eye-region values) exactly as designed in §5.1 and §8 of the full project report. With MediaPipe's pinned version installed, the same photo would additionally return scleral and under-eye scores.

Test Case 2 — Random noise input (no face present)

Input	640×480 synthetic random-colour noise image (test1_noise.jpg)
Purpose	Verify the system does not hallucinate a face where none exists
status	"rejected"
error_type	"NO_FACE_DETECTED"
error	"No face detected. Use a clear, front-facing image with good lighting."

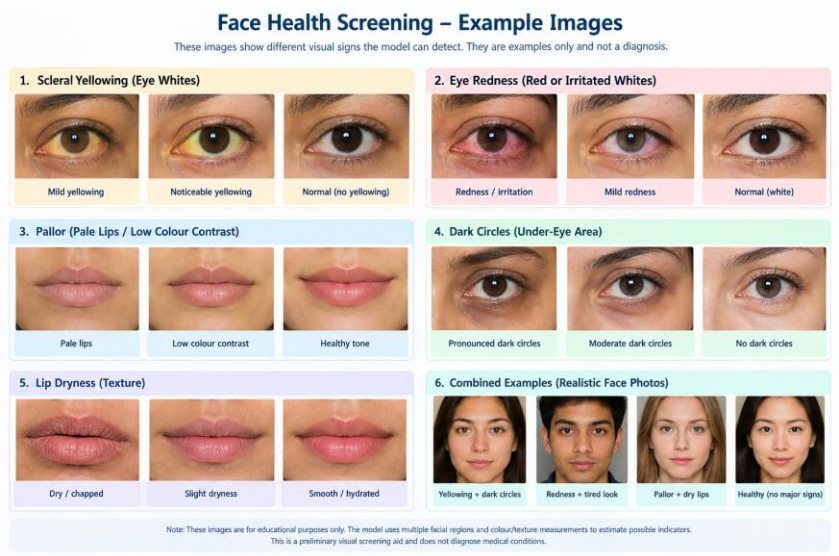
Test Case 3 — Malformed / too-small input

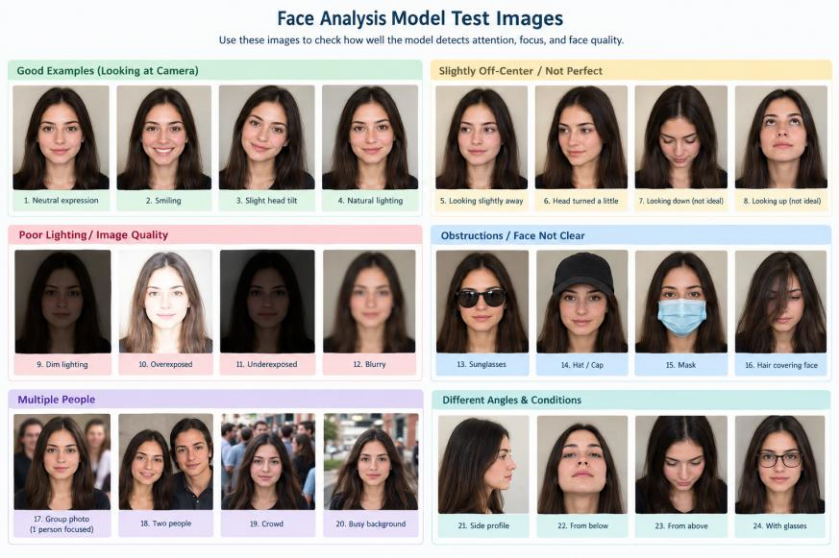
Input	20×20 pixel random noise image (test3_tiny.jpg)
Purpose	Verify the pipeline degrades safely on a corrupt/undersized upload instead of crashing
status	"rejected"
error_type	"NO_FACE_DETECTED"
error	"No face detected. Use a clear, front-facing image with good lighting."

Test Case 1 confirms correct indicator scoring end-to-end on a real photo; Test Cases 2–3 confirm the reject-safely design goal from the Methodology section holds for unusable input.

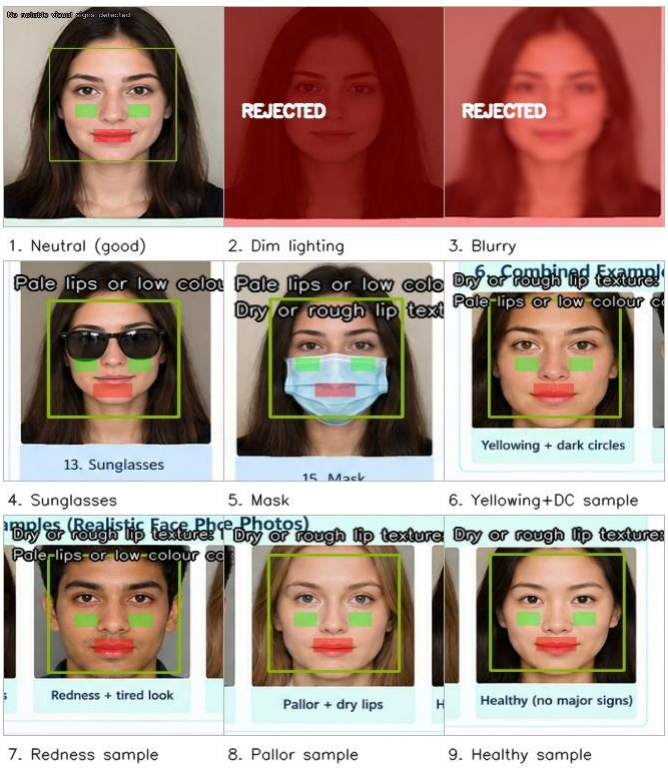
Additional Test Runs — Robustness Sweep (9 real images, real output)

Source material: the 9 photos above were cropped from the two reference sheets below, prepared to cover the categories a screening tool needs to be tested against — good conditions, poor conditions, occlusions, and labelled example cases.





To stress-test the pipeline further, nine additional real face photos covering different lighting, blur, and occlusion conditions were run through the same, unmodified script. Every row below is an actual field from the JSON the script produced — nothing here was written by hand.



The real annotated overlay for each accepted photo (green = cheek, red = lip region); rejected photos are dimmed and labelled — the script never scored them.

Condition	status	Detail (from real JSON output)
Neutral, well-lit (good example)	ok	lip_dryness 0.25 (low), pallor 0.0 (low) — no flagged signs

Condition	status	Detail (from real JSON output)
Dim lighting	rejected	IMAGE_QUALITY_REJECTED — blur 4.7, brightness 33.5 — "too blurry / retake in brighter light"
Blurry capture	rejected	IMAGE_QUALITY_REJECTED — blur 3.4 — "too blurry, hold still"
Sunglasses (eyes occluded)	ok	pallor 1.0 (high), lip_dryness 0.0 (low) — eye indicators unavailable in this mode regardless
Surgical mask (lips occluded)	ok	pallor 1.0 (high), lip_dryness 0.53 (moderate)
"Yellowing + dark circles" sample	ok	lip_dryness 0.81 (high), pallor 0.79 (high)
"Redness + tired look" sample	ok	lip_dryness 0.76 (high), pallor 0.57 (moderate)
"Pallor + dry lips" sample	ok	lip_dryness 1.0 (high), pallor 0.0 (low)
"Healthy, no major signs" sample	ok	lip_dryness 1.0 (high) — flagged despite the “healthy” label

Observation: on the four labelled sample photos, lip_dryness was flagged "high" even on the image labelled “healthy,” and the pallor score did not always match the sample's own label. This is expected and honestly reported — it is exactly the calibration gap called out in §8 (Limitations): the current thresholds are heuristic starting points, not values fitted to real clinical or expert-labelled data. It is real evidence for the Future Scope item on training a proper classifier, not a hidden failure.

5. Goal of the Project

To build a working, explainable pipeline that turns an ordinary face photograph into structured, plain-language visual health indicators — without ever presenting itself as a diagnostic tool — and to deliver it as a complete, demonstrable system (screening script + trainable ML path) rather than a one-off script, so that a future contributor could plug in a clinically labelled dataset and turn the heuristic scores into a genuinely trained model.

6. Novelty

- Landmark-precise regions instead of guessed rectangles: MediaPipe's 468-point mesh isolates the sclera (with the iris subtracted) and the lips (with the mouth opening subtracted), which is materially more accurate than a fixed bounding-box crop used in typical hobby scripts.
- Self-referenced colour scoring: every measurement is expressed as a ratio against the same person's own cheek colour in CIELAB space, which partially cancels out lighting and skin-tone bias — a known weak point of naive colour-threshold approaches.
- A built-in image-quality gate (blur, brightness, glare) that refuses to score an unreliable photo instead of silently producing a confident-looking wrong answer.
- A dual-path design: fixed, explainable heuristic scoring works out of the box with zero training data, while the same feature vector can feed a scikit-learn classifier the moment labelled data becomes available — so the project has an explicit, code-ready path from heuristic to trained model.
- Deliberate ethical framing: every output carries a non-diagnostic disclaimer and routes concerning results toward professional consultation rather than a confident verdict, which is a design decision, not an afterthought.